

TASK 3

MUSIC GENERATION WITH AI

Complete Project Task / Mini Project Report

Project Objective: Train a deep-learning model on MIDI note sequences and generate new musical sequences that can be saved as MIDI.

Domain	Artificial Intelligence / Deep Learning
Input	MIDI music files
Preprocessing	music21 note/chord extraction
Model	LSTM-based Recurrent Neural Network
Output	Generated MIDI music

Complete implementation, training workflow, testing and future enhancements.

1. Introduction

Music Generation with AI is the process of teaching a machine-learning model to learn patterns in musical data and then create new music. In this project, MIDI files are used as the training data. MIDI stores musical information such as notes, pitches and durations rather than recorded audio.

The project extracts musical events from MIDI files using the music21 library. These events are converted into sequences of tokens and used to train an LSTM (Long Short-Term Memory) neural network. After training, the model predicts the next musical event repeatedly to create a new sequence. The generated sequence is converted back into MIDI and can be played or rendered to audio using suitable software.

2. Objectives

- Collect a MIDI dataset containing musical pieces such as classical or jazz music.
- Extract notes and chords from MIDI files using music21.
- Convert musical events into numerical sequences suitable for deep learning.
- Build an LSTM-based RNN to learn musical patterns.
- Train the model using the prepared note sequences.
- Generate new musical sequences from a seed sequence.
- Convert the generated sequence into a MIDI file and save it.
- Optionally render the MIDI to audio for listening.

3. Technologies Used

- Python – main programming language.
- music21 – reading, analyzing and creating MIDI/music structures.
- NumPy – numerical array processing.
- TensorFlow/Keras – LSTM deep-learning model.
- MIDI-compatible player or synthesizer – to play generated music.
- Optional: FluidSynth or another MIDI renderer for WAV/MP3 conversion.

4. Dataset Collection

The first step is to collect MIDI files from a legally usable dataset. The dataset can contain classical, jazz, piano or other instrumental MIDI pieces. All files should be organized in one folder, for example **midi_dataset/**. The project should use music for which the user has permission or an appropriate license.

Example folder structure:

```
music-ai-project/  
  ■■■ midi_dataset/  
    ■ ■■■ piece01.mid  
    ■ ■■■ piece02.mid  
    ■ ■■■ piece03.mid  
    ■ ■■■ ...  
  ■■■ output/  
  ■■■ music_generator.py
```

5. Preprocessing with music21

MIDI files contain notes, chords, timing and other musical events. For a simple melody-generation project, notes and chords can be converted into symbolic tokens. A single note can be represented by its pitch name, while a chord can be represented by the pitches contained in the chord.

- Read each MIDI file.
- Parse the score using music21.
- Extract notes and chords.
- Convert each musical event to a string token.
- Combine tokens from the training files.
- Create a vocabulary of unique tokens.
- Convert tokens into integer IDs.
- Create input sequences and next-token targets.

6. Sequence Preparation

If the sequence length is 100, the model receives 100 musical events and learns to predict the 101st event. A sliding-window method is used to create many training examples from the same music sequence.

Example: [event1, event2, ..., event100] → target = event101

7. Complete Python Implementation

Install the required libraries before running the program:

```
pip install music21 tensorflow numpy
```

7.1 Data Collection and Note Extraction

```
from pathlib import Path
from music21 import converter, note, chord

DATA_DIR = Path("midi_dataset")

def extract_notes(midi_file):
    score = converter.parse(midi_file)
    notes = []

    for element in score.flatten().notes:
        if isinstance(element, note.Note):
            notes.append(str(element.pitch))
        elif isinstance(element, chord.Chord):
            notes.append(".".join(str(p) for p in element.normalOrder))

    return notes

all_notes = []

for midi_file in DATA_DIR.glob("*.mid"):
    try:
        events = extract_notes(midi_file)
        all_notes.extend(events)
        print("Processed:", midi_file.name, len(events))
    except Exception as e:
        print("Skipped:", midi_file.name, e)

print("Total musical events:", len(all_notes))
```

7.2 Create Training Sequences

```
import numpy as np

SEQUENCE_LENGTH = 100

unique_notes = sorted(set(all_notes))
note_to_int = {n: i for i, n in enumerate(unique_notes)}
int_to_note = {i: n for n, i in note_to_int.items()}

network_input = []
network_output = []

for i in range(len(all_notes) - SEQUENCE_LENGTH):
    sequence = all_notes[i:i + SEQUENCE_LENGTH]
    target = all_notes[i + SEQUENCE_LENGTH]

    network_input.append([note_to_int[n] for n in sequence])
    network_output.append(note_to_int[target])

n_vocab = len(unique_notes)

network_input = np.reshape(
    network_input, (len(network_input), SEQUENCE_LENGTH, 1)
)
network_input = network_input / float(n_vocab)

network_output = np.array(network_output)

print("Vocabulary size:", n_vocab)
print("Training sequences:", len(network_input))
```

7.3 Build the LSTM Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Dense, Activation

model = Sequential()

model.add(
    LSTM(
        512,
        input_shape=(SEQUENCE_LENGTH, 1),
        return_sequences=True
    )
)
model.add(Dropout(0.3))

model.add(LSTM(512))
model.add(Dropout(0.3))

model.add(Dense(256))
model.add(Activation("relu"))
model.add(Dropout(0.3))

model.add(Dense(n_vocab))
model.add(Activation("softmax"))

model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam"
)

model.summary()
```

7.4 Train and Save the Model

```
EPOCHS = 50
BATCH_SIZE = 64

model.fit(
    network_input,
    network_output,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE
)

model.save("music_lstm_model.keras")
print("Model saved successfully.")
```

8. Music Generation

After training, a random seed sequence is selected from the dataset. The model predicts the next event. The predicted event is added to the sequence, the window moves forward, and the process repeats until the required number of events is generated.

8.1 Generate New Musical Events

```
from tensorflow.keras.models import load_model
import random

model = load_model("music_lstm_model.keras")

START = random.randint(0, len(network_input) - 1)
pattern = list(network_input[START].flatten())

prediction_output = []

for _ in range(200):
    prediction_input = np.reshape(
        pattern[-SEQUENCE_LENGTH:],
        (1, SEQUENCE_LENGTH, 1)
    )

    prediction_input = prediction_input / float(n_vocab)

    prediction = model.predict(
        prediction_input, verbose=0
```

```
)  
  
index = np.argmax(prediction)  
result = int_to_note[index]  
  
prediction_output.append(result)  
pattern.append(index)  
  
print("Generated events:", len(prediction_output))
```

8.2 Convert Generated Events to MIDI

```
from music21 import stream, note, chord, instrument

output = stream.Stream()
output.append(instrument.Piano())

for event in prediction_output:
    if "." in event:
        pitches = [int(x) for x in event.split(".")]
        new_chord = chord.Chord(pitches)
        new_chord.quarterLength = 0.5
        output.append(new_chord)
    else:
        new_note = note.Note(event)
        new_note.quarterLength = 0.5
        output.append(new_note)

output.write("midi", fp="output/generated_music.mid")
print("Generated MIDI saved.")
```

9. Playing or Converting the MIDI

The generated MIDI can be opened in a MIDI-compatible music player or digital audio workstation. It can also be rendered to an audio file using a software synthesizer such as FluidSynth, provided it is installed and configured with a suitable soundfont.

Example workflow:

```
generated_music.mid
    ↓
MIDI player / synthesizer
    ↓
Audio playback
```

Optional:

```
generated_music.mid → FluidSynth + SoundFont → generated_music.wav
```

10. Complete Project Flow

MIDI Dataset → music21 Preprocessing → Note/Chord Tokens → Integer Encoding → Sequence Creation → LSTM Training → Next-Event Prediction → Generated Sequence → MIDI File → Audio Playback

11. Model Training

The LSTM learns relationships between previous musical events and the next event. During training, the input sequences are passed through the network and the model adjusts its weights to reduce prediction error. More training data and suitable hyperparameters can improve the model's ability to learn musical patterns, although excessive training can also cause overfitting.

Important parameters:

- Sequence length: number of previous events used as context.
- LSTM units: capacity of the recurrent layers.
- Dropout: helps reduce overfitting.
- Epochs: number of complete passes through the training data.
- Batch size: number of sequences processed before a weight update.
- Learning rate: controls the optimizer's step size.

12. Testing

Test	Expected Result
Load a valid MIDI file.	music21 parses the file successfully.
Extract notes/chords.	Musical events are stored as tokens.
Create sequences.	Input sequences and target events are generated.
Train the LSTM.	Training loss is produced and model is saved.
Generate music.	A new sequence of musical events is produced.
Write MIDI.	generated_music.mid is created.
Play/render MIDI.	Generated music can be heard or converted to audio.

13. Expected Output

- A trained file such as **music_lstm_model.keras**.
- A generated MIDI file such as **output/generated_music.mid**.
- A newly generated sequence that follows patterns learned from the training data.
- Optional audio output after MIDI rendering.

14. Advantages

- Can generate new music automatically.
- Learns musical patterns from existing MIDI data.
- MIDI output is compact and easy to edit.
- The approach can be adapted to different musical styles.
- The project demonstrates practical use of deep learning and sequence modeling.

15. Limitations

- Training can require substantial time and computing resources.
- The quality of generated music depends strongly on the dataset.
- A simple note-only representation does not capture every aspect of music, such as expressive performance and detailed rhythm.
- The generated music may contain repetition or lack long-term musical structure.
- Audio rendering requires an appropriate synthesizer and soundfont.

16. Future Enhancements

- Use larger and more diverse MIDI datasets.
- Include note duration, velocity and timing information.
- Use bidirectional or deeper sequence models where appropriate.
- Experiment with Transformers or GAN-based music generation.
- Add style conditioning, such as classical or jazz.
- Create a web interface for generating and downloading music.
- Add MIDI-to-audio rendering and an in-browser player.

17. Conclusion

The Music Generation with AI project demonstrates a complete pipeline for symbolic music generation. MIDI files are collected and processed with music21, musical events are converted into training sequences, and an LSTM-based recurrent neural network learns patterns from the data. The trained model generates new sequences, which are converted back into MIDI and can be played or rendered as audio. The project provides a practical introduction to AI-based sequence generation and can be extended with richer musical features and modern deep-learning architectures.

18. Final Summary

Collect MIDI → Preprocess with music21 → Prepare Sequences → Build LSTM → Train Model → Generate New Sequence → Convert to MIDI → Play / Save Audio

This document covers the task requirements, complete workflow, implementation code, testing, limitations, and future enhancements.